

CS5351 软件工程 2024/25学年B学期测验

(150分钟)

学生姓名： _____
学号： _____
座位号： _____

本测验为开卷测验。

题号	得分
1	
2	
3	
4	
5	
总分	
等级	

1. 软件过程（20分， 每指出1项缺失内容并说明改进之处得4分）

你的团队采用以下“Scrum”流程开发一款软件应用。项目经理认为该流程缺失了Scrum的部分关键要素，但无法明确指出具体缺失内容。你的任务是回顾下述项目特征，向项目经理说明该流程所缺失的要素。

请指出5项缺失要素，并分别说明：若将这些要素纳入流程，会如何改进项目当前使用的软件过程。

案例分析1

该项目设有项目待办列表（project backlog）和任务看板（task board），包含了所有待实现的用户功能需求。

用户与项目团队共同对项目待办列表中的任务进行优先级排序。
开发人员完成当前任务后，最好选择项目待办列表中优先级更高的任务来执行。

- 每项已实现的任务均由对应用户进行审核与测试。
- 每月月末，项目经理挑选用户已批准的任务，纳入即将发布的版本中。

为避免出错，项目会开展需求验证（requirement validation）、设计审核（design review，在团队确定待审核设计方案前进行）、现代代码审核（modern code review，在代码开发完成后进行）及测试工作。

你可参考以下角度分析：

- 如何了解项目整体进度是否符合计划？
- 开发人员如何知晓某一周期内需交付的需求规模？
- 能否预测项目的完成时间？
- 项目待办列表中的条目是否涵盖了所有待交付内容？
- 待办列表条目的交付顺序是否符合Scrum流程要求？

缺失项1：未将项目划分为冲刺（Sprint），且每个冲刺未设置冲刺待办列表（Sprint Backlog）
改进效果：待办列表的进度可按冲刺周期对照计划进行跟踪，开发人员也能更清晰地了解进度。
缺失项2：项目未对每个待办列表条目的工作量（或故事点、所需时间）进行估算
改进效果：通过工作量估算，开发人员可在当前冲刺开始时自行预估能完成的故事点数量，使进度更具可预测性。
缺失项3：未统计开发速度（velocity）
改进效果：通过跟踪开发速度及已估算工作量的剩余待办列表条目，项目可预测完成所有待办条目的冲刺周期（例如，可预测最佳、最差、平均三种情况下的周期）。
缺失项4：项目未跟踪技术债务（technical debt）及非功能需求（严格来说，非功能需求不属于技术债务）
改进效果：识别并审核技术债务，将其纳入项目待办列表。 优化后的项目待办列表能更真实地估算项目完成时间，且可能需要对待办条目重新排序，以提升项目开发速度。
缺失项5：因未引入“冲刺”概念，未强制要求在当前冲刺中优先实现高优先级任务
改进效果：优先选择当前冲刺需完成的待办条目，而非优先选择冲刺外的待办条目。
【备用】缺失项__：

缺失项1：未将项目划分为冲刺（Sprint），且每个冲刺未设置冲刺待办列表（Sprint Backlog）
改进效果：

（助教评分说明：学生可将缺失要素拆分表述，每一个独立角度的缺失要素均按1项计分。流程缺失要素的答案不唯一，符合Scrum核心逻辑即可。）

2. 软件需求工程（20分）

(a) 假设A是一项系统级的核心功能需求，B是与A相关的非功能需求。请结合你的课程项目，给出一组A与B的示例，并说明为何B是A的非功能需求。（3分）

	示例1
A示例：	
B示例：	
说明：	

(b) 若收集了软件的非功能需求但未实现，这会对待开发软件的运行产生问题吗？（3分）
若未实现非功能需求，软件可能无法满足用户使用需求（例如，无法支撑预期业务目标），或违反软件的业务约束条件。

(c) 需求工程包含需求获取（elicitation）、需求规格说明（specification）、需求验证（validation）与需求协商（negotiation）四个环节。其中，需求获取与规格说明的目的是明确待开发系统需解决的问题，并对其进行描述。（8分）

(1) 为何需要需求验证与需求协商？

需求验证：
通过与客户协同验证，确保需求的有效性。
需求协商：
需求协商可明确并调整需求范围，平衡功能需求与非功能需求的关系。

(2) 若仅听取干系人的需求、记录需求内容，并核对自身对各项需求的理解是否准确，这是否遗漏了上述四个环节中的部分内容？请举例说明：若按此方式收集需求，可能会产生哪些负面影响？
上述场景仅涵盖了需求获取、需求规格说明及部分需求验证环节，遗漏了以下内容：

- 需求协商环节；

- 需求验证环节中对所有需求的分析过程。

由此可能导致：非功能需求缺失、功能需求的一致性不明确、未平衡功能需求与非功能需求的关系，最终使需求规格说明不完整。

案例分析2

阅读案例分析2，回答下列问题。

约翰（John）计划为公司确定一款定制化调试器的需求，该调试器用于开发深度学习模型训练及样本推理的程序脚本。他从开发人员处收集了含缺陷的程序脚本及其修复版本，分析这些脚本存在的缺陷类型、运行时产生的错误，以及修复版本中用于确定含缺陷代码区域范围的关键信息，进而编写调试器的需求规格说明书。之后，他分别与每位开发人员逐一沟通调试器的界面草图，向开发人员展示该调试器将如何可视化错误、标注含缺陷代码区域，以便开发人员未来能更便捷地修复同类缺陷脚本。并根据开发人员的反馈，修改了规格说明书中的界面设计。

(d) 上述需求工程过程是否存在问题？请说明理由。（4分）

需求获取与需求验证过程存在问题。

当前需求由约翰单方面梳理，未充分结合开发人员的意见，可能无法反映开发人员在调试程序脚本时的实际需求；且验证过程仅针对界面设计，未验证调试器的核心功能是否符合需求。

(e) 假设约翰在协商过程中采用“社会相对主义”立场（即重视多方共识），请推荐一种需求获取技术以改进其流程，并详细说明推荐理由。（2分）

推荐的需求获取技术：需优先选择能引导开发人员达成共识的技术，例如（1）头脑风暴会议、（2）小组故事讲述法、（3）焦点小组法、（4）引导式研讨会。

推荐理由：上述技术均能通过群体互动促进开发人员充分沟通，帮助团队就调试器需求达成共识，符合社会相对主义立场的核心要求。

3. 软件架构（20分）

(a) 设计系统软件架构时，需考虑领域模型、设计、术语体系及架构呈现方式四个方面的因素。这些因素分别指什么？（4分）

- 领域模型：描述设计人员无法控制的环境约束，以及对架构有重要影响的需求。
- 设计：描述设计人员可控制的元素（即设计人员可自主决定这些元素的选择及排列方式）。
- 术语体系：软件架构需传达系统的非功能特性以满足用户非功能需求，因此应采用用户熟悉的术语，提升沟通效率。
- 呈现方式：可采用任何用户认为有效的呈现方式，无固定形式。

(b) 软件的质量属性（QA）数量较多，难以在常规项目中全面记忆、考虑并应用。应如何识别系统的核心质量属性？（4分）

首先，需确保需求中涵盖系统的非功能行为及核心功能需求（如核心用例）；

其次，尽可能使这些需求的非功能特性可度量；

最后，为这些需求分配对应的质量属性，并按质量属性对需求进行分组，筛选出覆盖核心需求的质量属性作为核心质量属性。

(c) 属性驱动设计（ADD）是一种在系统架构设计中考虑非功能质量属性的方法学。在软件架构教程中，我们会为架构元素选择在目标质量属性上表现良好的技术家族（或单一技术），以满足该元素的质量属性需求。教程案例中，我们通过三个子过程完成ADD应用：为架构元素（如迭代3中的批处理层 Batch Layer）识别质量属性、确定相关技术家族/技术、评估这些技术家族/技术。（12分）

(1) 若在应用ADD方法学时，每个架构元素仅能确定一个技术家族，且该技术家族下仅有一种可选技术，ADD方法学是否仍能有效应用？

☐ 同意 ☐ 不同意

(2) 若某架构元素在目标质量属性上的表现（得分）无法通过唯一可选的技术家族/技术得到满意解决，这会对需求与实现产生哪些影响？

- 对需求的影响：选择该技术家族/技术将可能导致系统设计无法满足需求中的质量属性要求。
- 对实现的影响：系统非功能需求的最终实现难度会增加（或导致项目成本上升、周期延长）。

(3) 若当前设计无法很好地满足架构元素的质量属性需求，应采取哪些措施？举例说明即可。

可从以下角度思考并作答：

- 当前架构元素的可选设计方案是否过于有限？
- 是否因高层架构元素分解为当前架构元素的方式不合理，导致当前元素难以满足质量属性需求？

若当前设计不适合该架构元素，应寻找其他解决方案：若现有方案均不理想，可针对该架构元素研究其他潜在技术方案；

或回到更高一层的架构抽象层面，分析是否存在其他可行的系统分解方式，使分解后的架构元素能更好地满足质量属性需求。例如，若某数据处理模块无法通过现有技术满足“高吞吐量”质量属性，可尝试将该模块拆分为“数据接收”“数据预处理”“数据计算”三个子模块，分别选择在“吞吐量”上表现良好的技术，再整合为新的架构元素。

4. 技术债务（20分）

本题结合技术债务教程中的案例（开发测试系统）作答。

(a) 若为软件系统选择了一套勉强可行但与整体设计目标契合度较低的软件架构（如技术债务教程中的案例架构），该选择是否会产生技术债务？（8分）

☐ 同意

☐ 不同意

若判定为技术债务，请按技术债务的四个维度对其进行分类，并说明分类理由。

维度	分类结果及理由
类型	设计债务（Design Debt）。理由：该问题源于架构设计选择不当，属于设计层面的债务。
意向性	有意债务（Intentional Debt）。理由：选择仅勉强可行的架构是主动决策，而非意外导致。
时间范围	长期债务（Long-term Debt）。理由：该决策会使系统中出现大量衍生的小债务，开发团队需长期处理其带来的后续影响。
影响范围	广泛影响型债务（Unfocused Debt）。理由：架构设计是抽象决策，会限制多个组件的底层设计方案，导致难以形成有效的解决方案，影响范围较广。

(b) 项目中存在三种典型的技术债务增加场景。结合第5讲教程中的案例（开发测试系统），分别给出“进度压力”与“代码重复”的示例，简要描述核心信息即可。（4分）

- 进度压力：案例背景提到，为加快可运行系统的交付速度，团队为某组件开发了“临时解决方案”（workarounds），该决策因进度优先而引入技术债务。
- 代码重复：在针对Oracle的“临时解决方案”设计中，团队决定从多个实现安全漏洞检测规则的开源项目中复制规则实现代码，将其封装为一组函数F（每个开源项目对应一个函数），并根据具体场景调用其中部分函数，此过程产生了代码重复。

(c) 案例研究中列出了七项技术债务。在采用【原始】设计方案开发最终解决方案前，应如何整体处理这些技术债务？请从项目待办列表角度给出三个示例。（8分）

处理策略：从项目待办列表角度对技术债务进行管理，将债务纳入待办流程跟踪。

示例1：识别债务（如标记为“债务X”），将其作为待办项纳入项目待办列表（并置于任务看板上）；监控返工量（如记录由债务引发的返工事件）；统计返工（含债务偿还）占项目预算的比例。

示例2：制定债务偿还计划，将“采用【偿还方案1】或【偿还方案2】偿还某类债务”作为待办项，安排在合适的开发周期中执行。

示例3：定期评估债务影响，将“季度技术债务评估”作为周期性待办项，根据评估结果调整债务优先级（如将高风险债务提前至近期冲刺中偿还）。

5. 现代代码审核（MCR）（20分）

案例分析3

阅读案例分析3，回答下列问题。

坎迪（Candy）在GitHub上为一个代码文件创建了分支，完成代码调试后，将修改后的文件上传至该分支，并在修改说明中简要标注了代码中的缺陷及修复方式。之后，她查看办公室情况，发现汤姆（Tom）和贝蒂（Betty）有空，便在GitHub上将这两位同事设为代码审核人。汤姆和贝蒂收到系统通知后，汤姆快速查看了修改内容及说明，通过系统批准了该修改；贝蒂因忙于其他任务，三天后才审核修改内容，最终也批准了修改，同时提出了一个更简洁的缺陷修复方案并说明理由。坎迪采用贝蒂的建议方案后，获得了贝蒂对新修改的批准，再经主管詹姆斯（James）最终检查批准，将修改提交到主分支。

(a) MCR具有非正式性、工具支撑性、异步性及聚焦补丁审核四大特征。案例分析3在哪些方面体现了这四大特征？请为每个特征各举一个示例。（10分）

特征	案例体现
非正式性	坎迪通过查看办公室内谁有空，非正式地确定了审核人名单。
工具支撑性	以GitHub为工具，系统自动向审核人发送通知，且修改内容、审核结果均记录在系统中。
异步性	两位审核人可选择不同时间段完成审核（汤姆即时审核，贝蒂三天后审核）。
聚焦补丁审核	审核人仅针对坎迪提交的代码修改内容（即补丁）进行审核。

(b) 贝蒂最初也批准了坎迪的原始修复方案。另一种可能的流程是：坎迪不采用贝蒂的建议方案，直接向詹姆斯提交原始方案的审批申请。假设詹姆斯看到两位审核人都已批准，便批准了原始方案。在该场景中，詹姆斯遗漏了MCR的哪项核心目的？詹姆斯应采取哪些行动以帮助团队获得更优方案？詹姆斯如何知晓可能存在更优的替代方案？若贝蒂的建议方案确实更优，坎迪应采取哪些行动？（10分）

核心目的	代码改进（Code Improvement）。注： 当前无法判断两种方案中哪一种更优，需进一步协商确认。
詹姆斯应采取的行动	不应直接批准修改，而应要求坎迪与贝蒂沟通，共同确定更优的修复方案；同时，詹姆斯需查看GitHub上的评论记录（贝蒂已在评论中提出建议方案及理由）。
詹姆斯如何知晓替代方案	通过查看GitHub上的审核评论，了解贝蒂提出的替代方案。

核心目的	代码改进 (Code Improvement)。注： 当前无法判断两种方案中哪一种更优，需进一步协商确认。
坎迪应采取的行动	若最终确定采用贝蒂的建议方案， 需将修改后的方案发送给两位审核人（或至少发送给贝蒂）， 待审核批准后再提交。